🗂️ Arrays — Master Revision Sheet (Step 3, P01–P41)

> **One-stop revision.** Each entry = 📖 **Problem** (what's asked + tiny example) + 🧾 **Recall** (the trigger) + 🔧 **Algorithm/idea** + ⚠️ **Trap**.

> Go top-to-bottom the night before an interview; every array problem in this step is here.

⚡ 60-Second Index

#	Problem	LC	Pattern	Time / Space
P01	Largest Element	179*	Linear scan	O(N) / O(1)
P02	Second Largest	GFG	Two-var single pass	O(N) / O(1)
P03	Check Sorted	1752*	Adjacent compare	O(N) / O(1)
P04	Remove Duplicates (sorted)	26	Slow/fast pointer	O(N) / O(1)
P05/06	Left Rotate by 1 / by D	189	Reversal algorithm	O(N) / O(1)
P07	Move Zeros to End	283	Write pointer + swap	O(N) / O(1)
P08	Linear Search	GFG	Scan	O(N) / O(1)
P09	Union & Intersection (sorted)	GFG	Two pointers	O(M+N) / O(1)
P10/13	Missing Number	268	XOR / Sum	O(N) / O(1)
P11	Max Consecutive Ones	485	Running counter	O(N) / O(1)
P12	Subarray with Given Sum	GFG	Sliding window	O(N) / O(1)
P14	Number Appearing Once	136	XOR cancel	O(N) / O(1)
P15	Search in 2D Matrix	74	Flattened binary search	O(log MN) / O(1)
P16	Row with Max 1s	GFG	Top-right staircase	O(M+N) / O(1)
P17	Two Sum	1	Hash complement	O(N) / O(N)
P18	Sort 0s/1s/2s	75	Dutch National Flag	O(N) / O(1)
P19	Majority > N/2	169	Boyer-Moore voting	O(N) / O(1)
P20/21	Max Subarray Sum	53	Kadane	O(N) / O(1)
P22	Stock Buy & Sell	121	Min-so-far	O(N) / O(1)
P23	Rearrange Alternating Signs	2149	Two write pointers	O(N) / O(N)
P24	Next Permutation	31	Dip → swap → reverse	O(N) / O(1)
P25	Leaders in Array	GFG	Suffix max	O(N) / O(1)

#	Problem	LC	Pattern	Time / Space
P26	Longest Consecutive Seq	128	Hash set, start-only	O(N) / O(N)
P27	Set Matrix Zeroes	73	First row/col as markers	O(MN) / O(1)
P28	Rotate Matrix 90°	48	Transpose + reverse rows	$O(N^2)$ / O(1)
P29	Spiral Matrix	54	4 shrinking boundaries	O(MN) / O(1)
P30	Pascal's Triangle	118/119	C(r,c) build	$O(N^2)$ / O(1)*
P31	Majority > N/3	229	2-candidate voting	O(N) / O(1)
P32/33	3Sum / 4Sum	15/18	Sort + two pointers	$O(N^2)$/$O(N^3)$
P34/35	0-Sum / XOR=K subarrays	GFG/LC	Prefix + hash map	O(N) / O(N)
P36	Merge Intervals	56	Sort by start, merge	O(N log N) / O(N)
P37	Merge Sorted Array	88	Three pointers from back	O(M+N) / O(1)
P38	Repeating & Missing	GFG	Sum & sum-of-squares	O(N) / O(1)
P39	Count Inversions	GFG	Merge sort count	O(N log N) / O(N)
P40	Reverse Pairs	493	Merge sort, count first	O(N log N) / O(N)
P41	Max Product Subarray	152	Track max & min	O(N) / O(1)

* variant / amortized

🧩 Pattern Toolbox (techniques that repeat)

- **Reversal trick** (P05/06): rotate by D = reverse[0,D-1] + reverse[D,N-1] + reverse all. `D %= N` first.
- **Slow/fast (write) pointer** (P04, P07, P18, P23): one pointer writes the "kept" region, other scans.
- **Boyer-Moore voting** (P19, P31): cancel out non-candidates; N/2 → 1 candidate, N/3 → 2 candidates, **always verify** when not guaranteed.
- **XOR magic** (P10, P14): `a^a=0`, `a^0=a` → pairs cancel, no overflow.
- **Prefix sum/XOR + hash map** (P12, P34, P35): "subarray with property" → store prefix, look up complement. Seed `map[0]=1`.
- **Sort + two pointers** (P09, P32, P33): collapse a nested loop into linear inner scan; skip duplicates.
- **Merge-sort-while-counting** (P39, P40): count cross-pairs during merge (or before, for Reverse Pairs).
- **Matrix as 1D** (P15): index `(mid/cols, mid%cols)` to binary-search a row-sorted matrix.
- **Staircase walk** (P16): start at a corner so each step eliminates a full row or column.

📒 Problem-by-Problem

P01 — Largest Element · O(N)/O(1)

- 📖 **Problem:** Find the biggest number in an array. `[3,2,1,5,6,4]` → `6`.
- 🧾 **Recall:** Init `maxVal = arr[0]`, update `maxVal = max(maxVal, arr[i])`.
- Linear scan; sorting is overkill. STL: `*max_element`.
- ⚠️ Don't init to `0` — fails on all-negative arrays.

P02 — Second Largest (no sort) · O(N)/O(1)

- 📖 **Problem:** Find the 2nd largest **distinct** value in one pass. `[12,35,1,10,34,1]` → `34`. All-equal `[10,10,10]` → `-1`.
- 🧾 **Recall:** Track `largest` & `second`. New largest → `second=largest, largest=x`. New second → `x>second && x!=largest`.
- Single pass, two variables. Return `-1` if no valid second.
- ⚠️ Must check `x != largest` or duplicates corrupt `second`.

P03 — Check if Sorted · O(N)/O(1)

- 📖 **Problem:** Is the array in non-decreasing (ascending, dups allowed) order? `[1,2,3,4,5]` →true, `[1,3,2,4]` →false.
- 🧾 **Recall:** Any `arr[i] > arr[i+1]` → false.
- ⚠️ Use `>` not `>=` for non-decreasing (`[1,1,2]` is sorted).

P04 — Remove Duplicates from Sorted Array · O(N)/O(1)

- 📖 **Problem:** Sorted array — delete duplicates in-place, return count of uniques. `[0,0,1,1,2,2,3,3,4]` → `5`, front = `[0,1,2,3,4,…]`.
- 🧾 **Recall:** `i=0` write ptr. For `j` from 1: if `arr[j]!=arr[i]` → `arr[++i]=arr[j]`. Return `i+1`.
- Works because sorted ⇒ duplicates are adjacent. In-place, no extra space.
- ⚠️ Start `j` at 1, not 0.

P05 / P06 — Left Rotate by 1 / by D · O(N)/O(1)

- 📖 **Problem:** Shift elements left. By 1: `[1,2,3,4,5]` → `[2,3,4,5,1]`. By D=2: `[1,2,3,4,5]` → `[3,4,5,1,2]`.
- 🧾 **Recall:** `d%=n`. reverse[0,d-1] + reverse[d,n-1] + reverse all.
- 🔧 **Algorithm (by D):** ① `d %= n` ② reverse first `d` ③ reverse last `n-d` ④ reverse whole array.
- By 1: save `arr[0]`, shift all left, place saved at end. Right-rotate by D ≡ left-rotate by N−D.
- ⚠️ Forgetting `d %= n` → out-of-bounds / wasted work.

P07 — Move Zeros to End · O(N)/O(1)

- 📖 **Problem:** Push all 0s to the end in-place, keep order of non-zeros. `[0,1,0,3,12]` → `[1,3,12,0,0]`.
- 🗒️ **Recall:** Write ptr `j=0`. If `arr[i]!=0` → `swap(arr[i], arr[j++])`.
- ⚠️ No extra array — must be in-place. Same write-pointer pattern as P04.

P08 — Linear Search · O(N)/O(1)

- 📖 **Problem:** Return index of `target`, or `-1`. `[2,3,4,10,40]`, t=10 → `3`.
- 🗒️ **Recall:** Scan L→R, return index on match, `-1` if exhausted.
- ⚠️ If array is sorted, interviewer likely wants Binary Search (O(log N)).

P09 — Union & Intersection of Sorted Arrays · O(M+N)/O(1)

- 📖 **Problem:** From two sorted arrays, get **Union** (all distinct) and **Intersection** (common). `A=[1,1,2,3,4,5], B=[2,3,4,4,5,6]` → Union `[1,2,3,4,5,6]`, Intersection `[2,3,4,5]`.
- 🗒️ **Recall:** Two pointers. **Union:** take smaller, skip dups, advance it. **Intersection:** advance both only on match.
- ⚠️ Also skip duplicates *within* a single array.

P10 / P13 — Missing Number (1..N) · O(N)/O(1)

- 📖 **Problem:** Array holds N−1 of the numbers 1..N; find the one missing. `[1,2,4,6,3,7,8]`, N=8 → `5`.
- 🗒️ **Recall:** `XOR(1..N) ^ XOR(arr)` → missing survives. (Or `N(N+1)/2 - sum`.)
- XOR avoids overflow; sum needs `long long` for large N.

P11 — Max Consecutive Ones · O(N)/O(1)

- 📖 **Problem:** Longest run of consecutive 1s in a binary array. `[1,1,0,1,1,1]` → `3`.
- 🗒️ **Recall:** `count = x==1 ? count+1 : 0`; track max.
- ⚠️ Update `maxCount` *before/while* resetting `count`, not after.

P12 — Subarray with Given Sum (positives) · O(N)/O(1)

- 📖 **Problem:** Non-negative array — find a contiguous subarray summing to `S`, return its bounds. `[1,2,3,7,5]`, S=12 → indices `[2,4]` (= `[2,3,7]`).
- 🗒️ **Recall:** Sliding window: expand right, shrink left while `sum > S`.
- 🔧 **Algorithm:** `l=0, sum=0`. For each `r`: `sum+=arr[r]`; while `sum>S`: `sum-=arr[l++]`; if `sum==S` → found `[l,r]`.
- ⚠️ **Only valid for non-negative arrays.** With negatives → prefix sum + hash map, O(N)/O(N).

P14 — Number Appearing Once (others twice) · O(N)/O(1)

- 📖 **Problem:** Every value appears twice except one; find the loner. `[4,1,2,1,2]` → `4`.
- 🧾 **Recall:** XOR all elements. Pairs cancel, single remains.
- ⚠️ Sorting is O(N log N) — XOR beats it.

P15 — Search in 2D Matrix · O(log MN)/O(1)

- 📖 **Problem:** Each row sorted, and every row's first > previous row's last (so the matrix reads as one sorted list). Is `target` present? `[[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, t=3 → true.
- 🧾 **Recall:** Treat as 1D sorted array; `val = matrix[mid/N][mid%N]`, binary search `[0, M*N-1]`.
- 🔧 **Algorithm:** `lo=0, hi=M*N-1`; `mid=(lo+hi)/2`; `val=matrix[mid/N][mid%N]`; standard binary search.
- ⚠️ `N` = number of **columns** (`mid/N` = row, `mid%N` = col).

P16 — Row with Max 1s (each row sorted) · O(M+N)/O(1)

- 📖 **Problem:** Binary matrix, each row sorted (0s then 1s). Return index of the row with the most 1s. Return `-1` if none. → row with leftmost first-1 wins.
- 🧾 **Recall:** Start top-right. See `1` → record row, `col--`. See `0` → `row++`.
- 🔧 **Algorithm:** `r=0, c=N-1, ans=-1`. While `r<M && c>=0`: if `mat[r][c]==1` → `ans=r, c--`; else `r++`.
- Each step kills a row or a column → linear. ⚠️ Top-left/bottom-right corners don't give this property.

P17 — Two Sum · O(N)/O(N)

- 📖 **Problem:** Return the two indices whose values add to `target` (exactly one solution). `[2,7,11,15]`, t=9 → `[0,1]`.
- 🧾 **Recall:** Hash map. For each `x`: if `target-x` in map → answer; else store `x→i`.
- ⚠️ Check complement **then** insert. Variant: sorted array + only yes/no → two pointers, O(1) space.

P18 — Sort 0s/1s/2s (Dutch National Flag) · O(N)/O(1)

- 📖 **Problem:** Array of only 0/1/2 — sort in-place, single pass. `[2,0,2,1,1,0]` → `[0,0,1,1,2,2]`.
- 🧾 **Recall:** `lo=mid=0, hi=n-1`. `0` →swap(lo,mid),lo++,mid++. `1` →mid++. `2` →swap(mid,hi),hi-- (don't move mid).
- 🔧 **Algorithm:** invariant — `[0,lo)`=0s, `[lo,mid)`=1s, `(hi,end]`=2s, `[mid,hi]`=unknown. Loop while `mid≤hi`.
- ⚠️ After swapping with `hi`, do **not** advance `mid` (unknown element just arrived).

P19 — Majority Element > N/2 · O(N)/O(1)

- 📖 **Problem:** Find the value occurring more than N/2 times (guaranteed to exist). `[2,2,1,1,1,2,2]` → `2`.
- 🧾 **Recall:** Boyer-Moore: candidate+count. Match → `count++`, else `count--`; `count==0` → new candidate.
- 🔧 **Algorithm:** intuition — the majority outnumbers everything else combined, so pairwise cancellation leaves it standing.
- ⚠️ Verify candidate in a second pass if majority isn't guaranteed.

P20 / P21 — Kadane's Max Subarray (sum / + indices) · O(N)/O(1)

- 📖 **Problem:** Max sum of a contiguous subarray (P21 also returns its start/end). `[-2,1,-3,4,-1,2,1,-5,4]` → `6` (`[4,-1,2,1]`). Empty subarray not allowed.
- 🧾 **Recall:** `currentSum = max(arr[i], currentSum + arr[i])`; track `maxSum`. (Reset on negative drag.)
- Indices: set `tempStart=i` on fresh start; commit `start=tempStart, end=i` on new max.
- ⚠️ Init `maxSum = arr[0]`, **not 0** (all-negative case). Circular variant (LC 918) = max(Kadane, total − minSubarray).

P22 — Stock Buy & Sell (single transaction) · O(N)/O(1)

- 📖 **Problem:** Buy one day, sell a later day, maximize profit. `[7,1,5,3,6,4]` → `5` (buy 1, sell 6); `[7,6,4,3,1]` → `0`.
- 🧾 **Recall:** Track `minPrice` so far; `maxProfit = max(maxProfit, price - minPrice)`.
- Compute profit with old min, then update min.

P23 — Rearrange Alternating Signs (equal counts) · O(N)/O(N)

- 📖 **Problem:** Reorder so signs alternate starting positive, preserving relative order. Equal +/− counts. `[3,1,-2,-5,2,-4]` → `[3,-2,1,-5,2,-4]`.
- 🧾 **Recall:** Positives → even idx (0,2,4...), negatives → odd idx (1,3,5...); step by 2.
- 🔧 **Algorithm:** `pos=0, neg=1`. For each x: if `x>=0` → `res[pos]=x, pos+=2`; else `res[neg]=x, neg+=2`.
- ⚠️ Different approach needed if counts unequal — read constraints.

P24 — Next Permutation · O(N)/O(1)

- 📖 **Problem:** Rearrange in-place to the lexicographically next-greater permutation; if largest (descending), wrap to smallest. `[1,2,3]` → `[1,3,2]`, `[3,2,1]` → `[1,2,3]`, `[1,1,5]` → `[1,5,1]`.
- 🧾 **Recall:** Rightmost dip `i` (`arr[i]<arr[i+1]`) → rightmost `arr[j]>arr[i]` → swap(i,j) → reverse suffix after `i`.
- 🔧 **Algorithm:** ① scan from right for first `i` with `arr[i]<arr[i+1]` ② from right find `j` with `arr[j]>arr[i]` ③ swap ④ reverse `arr[i+1…]`.

- ⚠️ No dip (fully descending) → just reverse the whole array.

P25 — Leaders in Array · O(N)/O(1)

- 📖 **Problem:** An element is a "leader" if greater than everything to its right (rightmost always is). `[16,17,4,3,5,2]` → `[17,5,2]`.
- 🧾 **Recall:** Scan **right→left**, keep `maxRight`. If `arr[i] > maxRight` → leader, update.
- ⚠️ Left→right checking all right elements is $O(N^2)$; reverse the result at the end to restore order.

P26 — Longest Consecutive Sequence · O(N)/O(N)

- 📖 **Problem:** Length of the longest run of consecutive integers (any order). Must be O(N). `[100,4,200,1,3,2]` → `4` (`1,2,3,4`).
- 🧾 **Recall:** Hash set. Only start counting where `num-1 ∉ set` (a sequence start); walk chain.
- 🔧 **Algorithm:** put all in set. For each `num` with `num-1` absent: `len=1`; while `num+len` in set: `len++`; update max.
- ⚠️ The "start-only" guard is what keeps it O(N); expanding from every element → $O(N^2)$.

P27 — Set Matrix Zeroes (in-place) · O(MN)/O(1)

- 📖 **Problem:** If a cell is 0, zero its entire row and column, in-place. `[[1,1,1],[1,0,1],[1,1,1]]` → `[[1,0,1],[0,0,0],[1,0,1]]`.
- 🧾 **Recall:** Use first row/col as markers (with separate flags for them). Mark → apply inner → zero first row/col last.
- 🔧 **Algorithm:** ① record if row0/col0 themselves contain a 0 (two flags) ② for inner cells, a 0 sets `mat[i][0]` & `mat[0][j]` ③ apply markers to inner cells ④ finally zero row0/col0 per flags.
- ⚠️ Zero the first row/col **after** applying markers, never before (cascades).

P28 — Rotate Matrix 90° Clockwise · $O(N^2)$/O(1)

- 📖 **Problem:** Rotate an N×N matrix 90° clockwise, in-place. `[[1,2,3],[4,5,6],[7,8,9]]` → `[[7,4,1],[8,5,2],[9,6,3]]`.
- 🧾 **Recall:** Transpose (swap `[i][j]↔[j][i]` for `j>i`) then reverse each row.
- 🔧 **Algorithm:** ① transpose upper triangle ② reverse every row. (Counter-clockwise: transpose then reverse each **column**.)
- ⚠️ Transpose only `j>i`, else you swap twice and undo it.

P29 — Spiral Matrix · O(MN)/O(1)

- 📖 **Problem:** Output all elements in clockwise spiral order. `[[1,2,3,4],[5,6,7,8],[9,10,11,12]]` → `[1,2,3,4,8,12,11,10,9,5,6,7]`.
- 🧾 **Recall:** 4 boundaries top/bottom/left/right. top→ right↓ bottom← left↑; shrink after each; guard `top≤bottom` & `left≤right`.

- 🔧 **Algorithm:** loop while `top≤bottom && left≤right`: walk top row (left→right), `top++`; right col (top→bottom), `right--`; if `top≤bottom` bottom row (right→left), `bottom--`; if `left≤right` left col (bottom→top), `left++`.
- ⚠️ Missing the two guards double-counts a middle row/column.

P30 — Pascal's Triangle (3 variants) · O(N²)/O(1) per row

- 📖 **Problem:** Triangle where each entry = sum of the two above; entry at row r, col c = `C(r,c)`. Variants: ① print N rows ② print Nth row ③ value at (R,C).
- 🧾 **Recall:** Element = `C(r,c)`. Build a row in O(N): `row[k] = row[k-1] * (n-k+1) / k`, start 1.
- Full triangle: `row[j] = above[j-1] + above[j]`.
- ⚠️ Numerator is `(n-k+1)`, not `(n-k)` — common off-by-one.

P31 — Majority Element > N/3 · O(N)/O(1)

- 📖 **Problem:** Find all values appearing more than N/3 times (**at most 2** can). `[1,1,1,3,3,2,2,2]` → `[1,2]`.
- 🧾 **Recall:** Two candidates, two counts. Match→++; both zero→assign; else both--. **Then verify both.**
- 🔧 **Algorithm:** extended Boyer-Moore with `cand1,cnt1,cand2,cnt2`; second pass counts real occurrences and keeps those `> N/3`.
- ⚠️ Verification pass is mandatory — voting candidates aren't guaranteed.

P32 / P33 — 3Sum / 4Sum · O(N²) / O(N³)

- 📖 **Problem:** Find all **unique** triplets summing to 0 (3Sum) / quadruplets summing to `target` (4Sum). `[-1,0,1,2,-1,-4]` → `[[-1,-1,2],[-1,0,1]]`.
- 🧾 **Recall:** Sort. **3Sum:** fix `i`, two pointers `l=i+1,r=n-1` for `-arr[i]`. **4Sum:** fix `i,j`, then two pointers; use `long long`.
- 🔧 **Algorithm:** sort first. For each fixed index, move `l,r` inward: `sum<target` → `l++`, `>` → `r--`, `==` →record & skip dups on both sides.
- ⚠️ Dup-skip guard is `if (i > 0 && arr[i]==arr[i-1]) continue;`.

P34 / P35 — Largest 0-Sum Subarray / Count XOR=K · O(N)/O(N)

- 📖 **Problem: P34:** length of the longest subarray summing to 0. `[15,-2,2,-8,1,7,10,23]` → `5`. **P35:** count subarrays whose XOR = K.
- 🧾 **Recall: P34:** same prefix sum seen again → zero-sum subarray; store **first** occurrence; len `= i - firstSeen[prefix]`. **P35:** `count += freq[xorPref ^ K]`, then `freq[xorPref]++`; seed `freq[0]=1`.
- 🔧 **Why it works:** if prefix[i]==prefix[j] the segment between them sums to 0. For XOR: a subarray (j..i) has XOR K iff `prefix[i] ^ prefix[j-1] = K` ⇒ `prefix[j-1] = prefix[i] ^ K`.

- ⚠️ P34 stores the **first** occurrence (max length); storing latest gives min length.

P36 — Merge Overlapping Intervals · O(N log N)/O(N)

- 📖 **Problem:** Merge intervals that overlap. `[[1,3],[2,6],[8,10],[15,18]]` → `[[1,6],[8,10],[15,18]]`.
- 🧾 **Recall:** Sort by start. If `curr.start ≤ last.end` → merge `last.end = max(last.end, curr.end)`; else push new.
- ⚠️ Must sort first; the overlap check assumes sorted starts.

P37 — Merge Sorted Array (in-place into nums1) · O(M+N)/O(1)

- 📖 **Problem:** `nums1` has `m` valid + `n` empty slots; `nums2` has `n`. Merge both sorted into `nums1` in-place. `[1,2,3,0,0,0],m=3` + `[2,5,6]` → `[1,2,2,3,5,6]`.
- 🧾 **Recall:** Three pointers from the back: `p1=m-1, p2=n-1, p=m+n-1`; place larger at `nums1[p]`. Drain remaining `nums2`.
- 🔧 **Algorithm:** while `p2>=0`: if `p1>=0 && nums1[p1]>nums2[p2]` → `nums1[p--]=nums1[p1--]` else `nums1[p--]=nums2[p2--]`.
- ⚠️ Merge from the **back** — front merge overwrites unread `nums1` values.

P38 — Find Repeating & Missing · O(N)/O(1)

- 📖 **Problem:** Numbers 1..N each once, except one repeats and one is missing — find both. `[4,3,6,2,1,1]` → repeat `1`, missing `5`.
- 🧾 **Recall:** `d1 = sum − N(N+1)/2 = R−M`; `d2 = sumSq − Σk² = R²−M²`; `R+M = d2/d1`. Solve the two equations.
- 🔧 **Algorithm:** from `R−M` and `R+M`: `R=((R-M)+(R+M))/2`, `M=R-(R-M)`. (XOR-bucket method is the alternative.)
- ⚠️ Use `long long` — sum of squares overflows.

P39 — Count Inversions · O(N log N)/O(N)

- 📖 **Problem:** Count pairs `(i<j)` with `arr[i] > arr[j]` (how far from sorted). `[2,4,1,3,5]` → `3`; fully reversed → `N(N-1)/2`.
- 🧾 **Recall:** Merge sort: when `right[j] < left[i]`, `count += (mid − i + 1)` (all remaining lefts invert).
- 🔧 **Algorithm:** standard merge sort; add the cross-count during the merge step.
- ⚠️ Count must be `long long` (can be ~$N^2/2$).

P40 — Reverse Pairs (`arr[i] > 2*arr[j]`) · O(N log N)/O(N)

- 📖 **Problem:** Count pairs `(i<j)` with `arr[i] > 2*arr[j]`. `[1,3,2,3,1]` → `2`; `[2,4,3,5,1]` → `3`.
- 🧾 **Recall:** Count **first** in a separate pass (`while arr[i] > 2*arr[j]: j++`), **then** merge normally.

- 🔧 **Algorithm:** in merge sort, before merging the two sorted halves, count pairs with a dedicated two-pointer scan; then do the ordinary merge.
- ⚠️ Conditions for counting and merging differ — you can't count during the merge like in inversions. Use `long long` for `2*arr[j]`.

P41 — Maximum Product Subarray · O(N)/O(1)

- 📖 **Problem:** Max product of a contiguous subarray. `[2,3,-2,4]` → `6`; `[-2,3,-4]` → `24` (two negatives flip to a big positive).
- 🧾 **Recall:** Track `maxProd` **and** `minProd`. `newMax = max(nums[i], maxProd*nums[i], minProd*nums[i])`. Save old max before updating min.
- 🔧 **Why min too:** a negative number swaps the roles — the smallest (most negative) product can become the largest. Zero resets both.
- ⚠️ Pure Kadane (max only) misses the negative-flip; you must keep min.

🎯 Last-Minute Trap Checklist

- [ ] Init max/min to `arr[0]`, never `0` (negatives). — P01, P20
- [ ] `d %= n` before rotating. — P05/06
- [ ] Sliding window ⇒ non-negative only; negatives need prefix+hash. — P12, P34/35
- [ ] DNF: don't advance `mid` after a `hi` swap. — P18
- [ ] Voting always needs a verify pass when not guaranteed. — P19, P31
- [ ] Merge sorted array & three-pointer tricks go **from the back**. — P37
- [ ] `long long` for sums, squares, inversion/pair counts. — P10, P38, P39, P40
- [ ] Matrix: transpose only `j>i`; spiral needs `top≤bottom` / `left≤right` guards. — P28, P29
- [ ] Max Product keeps **both** max and min. — P41
- [ ] Reverse Pairs: count BEFORE merging (condition ≠ merge condition). — P40

Generated from the per-problem notes in this folder (P01–P41). For full dry-runs, code, and interview scripts, open the individual `P##_*.md` files.